Nextflow from scratch: building and running reproducible pipelines

Sarai Varona
BU-ISCIII
Bioinformatics Unit, Institute of Health Carlos III
Madrid, Spain

Overview

1. Introduction to Nextflow
2. Hello Nextflow
3. From Nextflow to nf-core

HANDS-ON



https://github.com/svarona/nextflow-101-sebibc

1. Introduction to Nextflow

1.1. What is Nextflow



DSL (Domain Specific Language) → **Create workflows:**

- Scalable
- Portable
- Reproducible

Dataflow programming model → **Simplifies writing** parallel and distributed pipelines

Variety of **execution platforms** →

- Local machine
- HPC schedulers
- Cloud

1.2. Multiple execution environments

nextflow

Supports:

- Compute environments
- Software container runtimes
- Package managers






aws

seqera

2.Hello Nextflow

2.1. The anatomy of a Nextflow pipeline

PROCESS

Defines one or more computational steps.

Think: run FastQC

WORKFLOW

Connects the processes

Think:
fastp → bowtie2 → MultiQC

CHANNEL

Carries data between processes.

Think: The reference .fasta file path

2.2. Processes: defining a computational step

```
process SAY_HELLO {
    input:
    val greeting

    output:
    path 'output.txt'

    script:
    """
    echo '$greeting' > output.txt
    """
}
```

INPUT
Data required

SCRIPT
Command to be executed

OUTPUT
Captured results

2.2. Processes: defining a computational step

```
process SAY_HELLO {
    input:
    val greeting

    output:
    path 'output.txt'

    script:
    """
    echo '$greeting' > output.txt
    """
}
```

Strings or numbers

For files or directories.

- Path input: Nextflow stages it into the task's working directory
- Path output: Nextflow expects that file to be created in the task's working directory

2.2. Processes: defining a computational step

```
process SAY_HELLO {
    input:
    val greeting

    output:
    path 'output.txt'

    script:
    """
    echo '$greeting' > output.txt
    """
}
```



```
process SAY_HELLO {
    output:
    path 'output.txt'

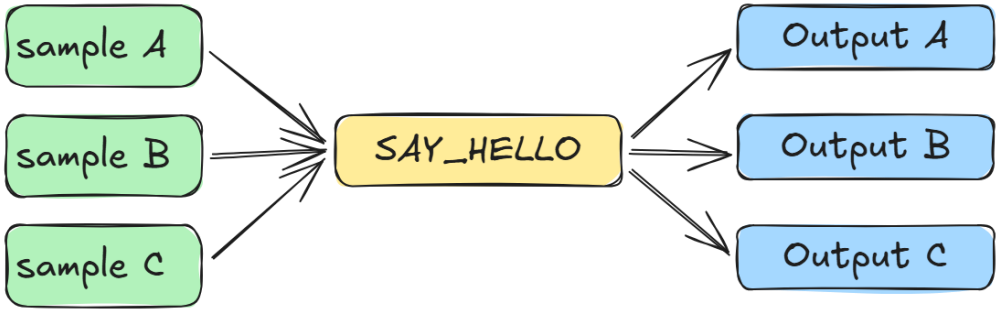
    script:
    """
    echo 'Hello world' > output.txt
    """
}
```

2.2. Processes: defining a computational step

```
process SAY_HELLO {
    input:
    val greeting

    output:
    path 'output.txt'

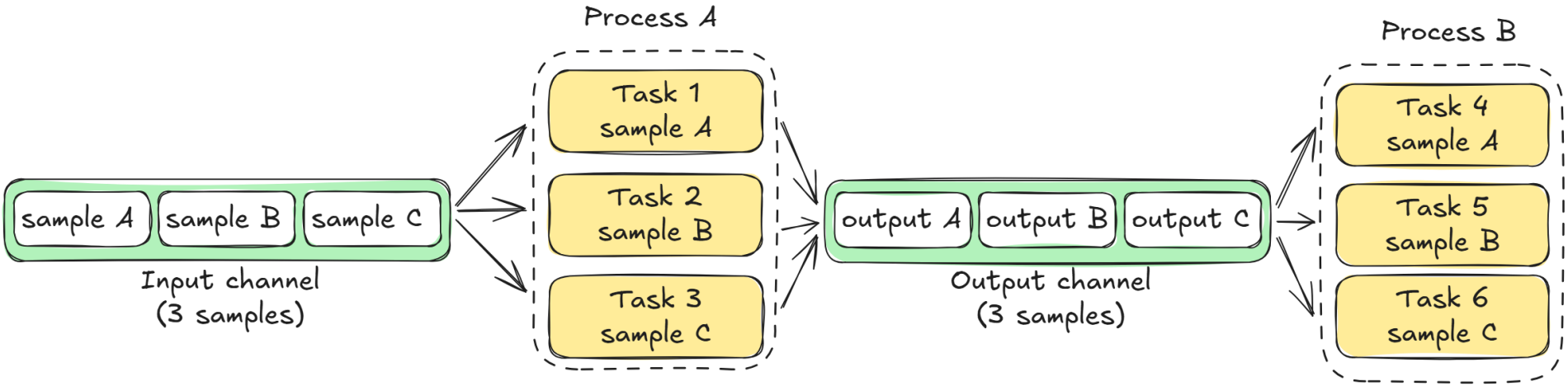
    script:
    """
    echo '$greeting' > output.txt
    """
}
```



- Runs the process with a set of inputs
- Nextflow create a separate task for each
- Independent tasks can run in parallel

2.3. Channels: moving data through a pipeline

A channel carries data → **Values** · **files** · **tuples / combinations**



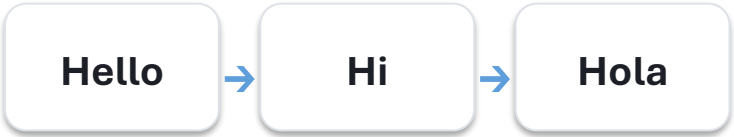
2.3. Channels: Channel factory

- **channel.of**: Creates a channel from one or more supplied values
- **channel.fromList()**: Creates a channel from a list
- **channel.fromPath()**: Creates a channel by matching files or directories using a path or glob pattern

```
workflow {
    greetings = channel.of('Hello','Hi','Hola')
    SAY_HELLO(greetings)
}

workflow {
    greetings = channel.fromList(['Hello','Hi','Hola'])
    SAY_HELLO(greetings)
}
```

3 independent task executions



2.4. Operators: transforming channel contents

Operators: transform or reorganise channel's content

.map(): Transforms each item into a new value

```
channel
    .fromPath('data/*.fastq')
    .map { file -> file.baseName }
```

.filter(): Keeps only items that match a condition

.collect(): Gathers all items into a single collection

.flatten(): Unpacks collections emitted by a channel into individual elements

Operators can be chained to build data transformations

2.4. Operators: transforming channel contents

Operators: transform or reorganise channel's content

.map(): Transforms each item into a new value

.filter(): Keeps only items that match a condition

.collect(): Gathers all items into a single collection

.flatten(): Unpacks collections emitted by a channel into individual elements

Operators can be chained to build data transformations

```
channel
    .fromPath('data/*.fastq')
    .map { file -> file.baseName
}

workflow {
    greetings = channel
      .of(['Hi','Hola'])
      .flatten()
    SAY_HELLO(greetings)
}
```

2 independent task executions

2.4. Operators: Debugging channels

println: Display values

```
Println “Input parameter: ${params.input}”
```

- Prints variables, parameters and other values
- Does not require a channel

.view(): Inspect channel contents

```
ch_names.view { “Input name: $it” }
```

- Prints each element emitted by a channel
- Useful for inspecting intermediate results

2.4. Operators: Debugging channels

println: Display values

```
Println “Input parameter: ${params.input}”
```

- Prints variables, parameters and other values
- Des not require a channel

.view(): Inspect channel contents

```
ch_names.view { “Input name: $it” }
```

- Prints each element emitted by a channel
- Useful for inspecting intermediate results

```
println ch_names

DataflowBroadcast around DataflowStream…
```

2.5. Workflows: connecting everything together

```
workflow sayHello {
    take:
    ch_greetings

    main:
    SAY_HELLO(ch_greetings)

    emit:
    results = SAY_HELLO.out
}
```

TAKE
Defines the inputs
(channels)

MAIN
Workflow logic
(Call processes)

EMIT
Defines the outputs
(available for other workflows)

2.5. Workflows: connecting everything together

```
workflow sayHello {
    take:
    ch_greetings

    main:
    SAY_HELLO(ch_greetings)

    emit:
    results = SAY_HELLO.out
}
```

```
process SAY_HELLO {
    input:
    val greeting

    output:
    path 'output.txt'

    script:
    """
    echo '$greeting' > output.txt
    """
}
```

2.6. Pipeline parameters and input channels

Define and access parameters

In the command line

```
nextflow run main.nf --input 'Alice'
```

In the nextflow workflow:

```
ch_names = channel.of(params.input)
```

Pass the channel to a process:

```
SAY_HELLO(ch_names)
```

2.7. emit vs publish: Reusing workflows and exposing results

***emit* | Workflow composition**

- When you want the outputs to be reused by other workflows
- Defines the outputs that a workflow makes available to its caller
- Outputs are accessed through the workflow interface (workflow_name.out)

***publish* + *output* | User-facing results**

- When you want the final results available to the user in an output directory
- *publish*: selects which channels from the entry workflow should be published
- output {} defines how and where those results are published
- Results are written to the output directory

2.7. Workflows: connecting everything together

subworkflows

```
workflow createGreetings {
    take:
    names
    main:
    greetings = names.map { name -> "Hello ${name}!" }
    emit:
    results = greetings
}
workflow saveGreetings {
    take:
    greetings
    main:
    SAVE_GREETING(greetings)
    emit:
    files = SAVE_GREETING.out
}
```

Entry workflow

```
workflow {
    main:
    names = channel.of('Alice', 'Bob', 'Charlie')
    createGreetings(names)
    saveGreetings(createGreetings.out.results)

    publish:
    greeting_files = saveGreetings.out.files
}
output {
    greeting_files {
        path 'greetings'
    }
}
```

3.1. Nextflow CLI options vs. pipeline parameters

Nextflow options (-)

Control how Nextflow executes the workflow.

Example:

```
-resume
-output-dir results
```

Pipeline parameters (--)

Define the inputs and configuration of the pipeline.

Example:

```
--input samples.csv
--outdir results
```

3.1. Nextflow CLI options vs. pipeline parameters

Nextflow options (-)

Control how Nextflow executes the workflow.

Example:

```
-resume
-output-dir results
```

Pipeline parameters (--)

Define the inputs and configuration of the pipeline.

Example:

```
--input samples.csv
```

```
nextflow run main.nf \
    --input samples.csv \
    -resume \
    -output-dir results
```

3.2. Common Nextflow execution options

-resume

Resume a previous execution

- Reuses previously cached task executions when possible
- Avoids repeating successfully completed tasks if their cache is valid

-output-dir

Specify the output directory.

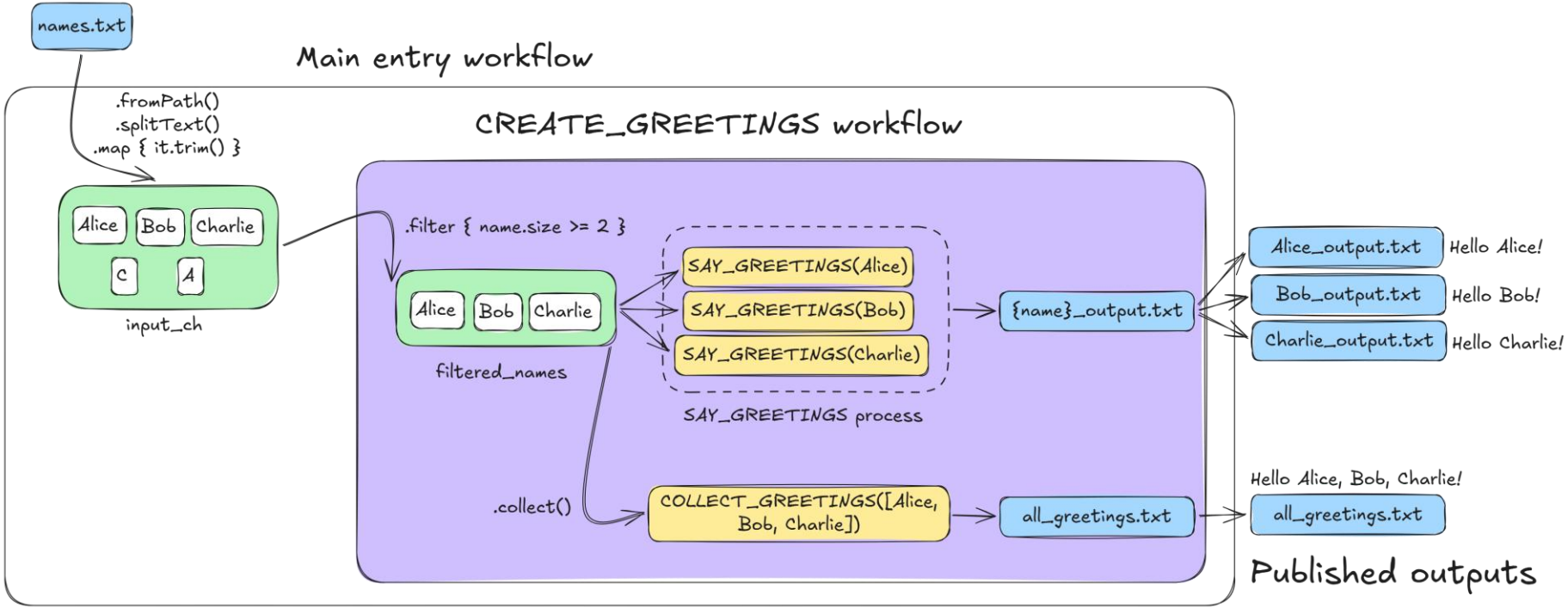
- Defines where published workflow outputs are stored
- Overrides the default output directory (results)

HANDS-ON



https://github.com/svarona/nextflow-101-sebibc

Hands-on schema



3. From Nextflow to nf-core

3.1. What is nf-core?

- Project (2018) Phil Ewels
- Standards pipelines
- Outgrow
- nf-core community
- Voluntary
- Projects&Grants

https://nf-co.re/

3.1. What is nf-core?

An open-source community for building reproducible Nextflow workflows

COMMUNITY

People collaborating on workflows and reusable components

STANDARDS

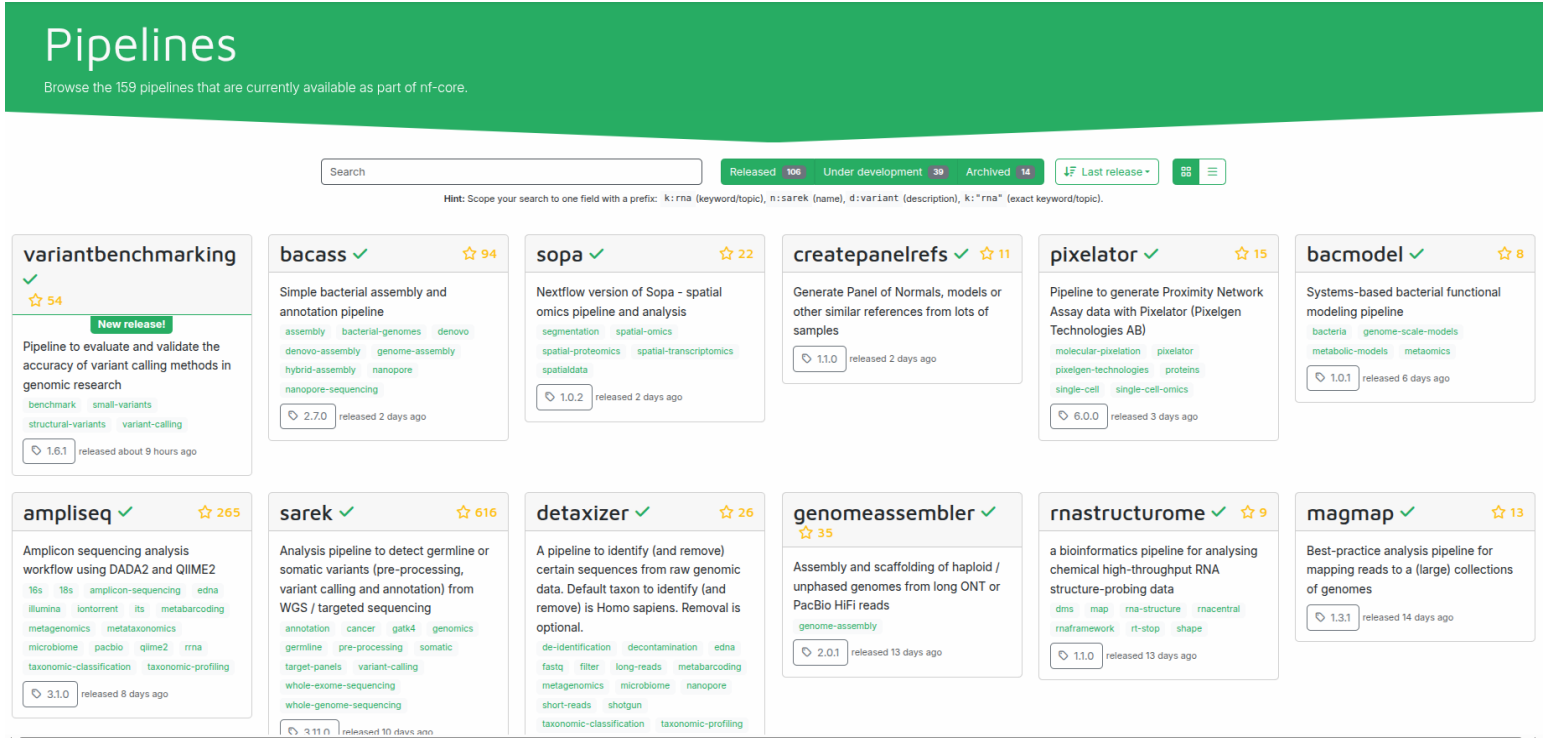
Shared best practices, specifications and templates

REUSABILITY

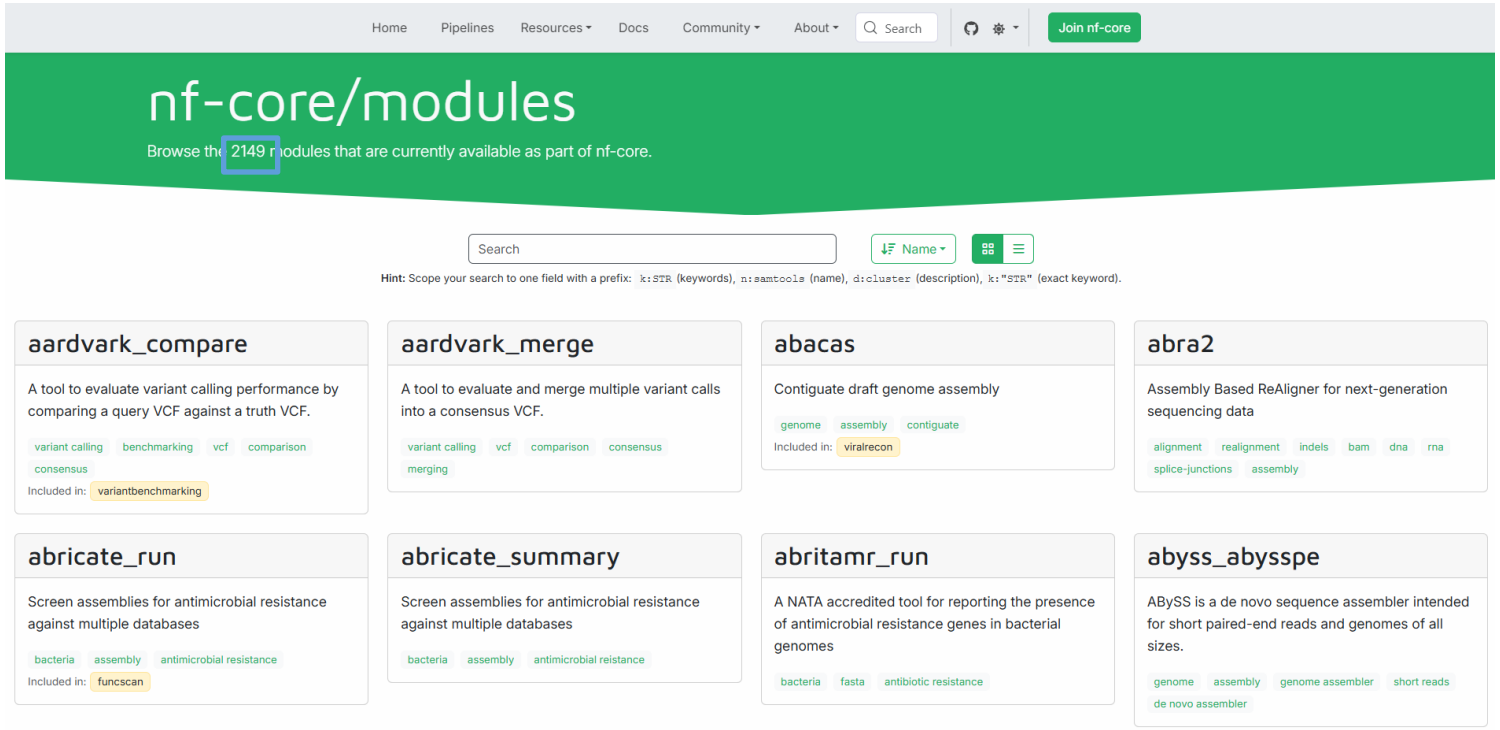
Modules and subworkflows shared across pipelines

nf-core = community + standards + reusable Nextflow components + pipelines

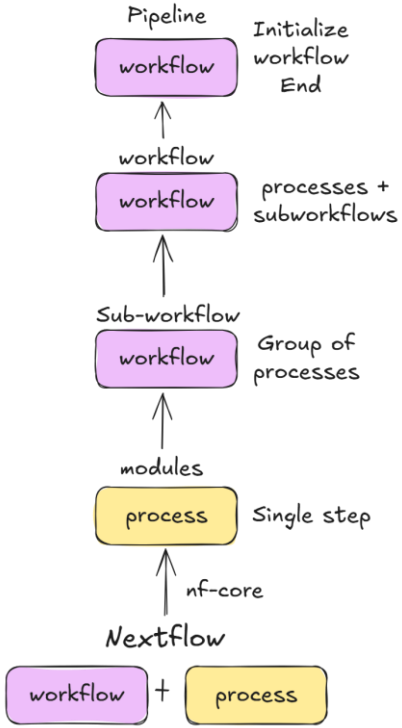
3.1. What is nf-core?



3.1. What is nf-core?



3.2. From individual workflows to a community ecosystem



Shared across the community

- Standardised structure
- Reusable components
- Testing
- Documentation
- Peer review
- Continuous integration

Build once → reuse everywhere

3.3. Modules

```
fastqc/
├── environment.yml
├── main.nf
│   └── process FASTQC { ... }
├── meta.yml
└── tests
    ├── main.nf.test
    └── main.nf.test.snap

2 directories, 5 files
```

Reusable wrappers for individual tools

- One process (1/+ tools)
- Standardised inputs and outputs
- Software dependencies and version reporting
- Documentation and tests

Write the wrapper once, reuse it across pipelines.

3.4. Subworkflows

```
fastq_align_bowtie2/
├── main.nf
    └── workflow FASTQ_ALIGN_BOWTIE2 {...}
        └── BOWTIE2_ALIGN(ch_reads, ch_index)
        └── ALIGN_STATS(BOWTIE2_ALIGN.out.bam)
├── meta.yml
└── tests
    ├── main.nf.test
    ├── main.nf.test.snap
    └── nextflow.config
```

Reusable combinations of processes

- Combine multiple modules
- Encapsulate a common analysis step
- Reusable across pipelines

Modules are building blocks.

Subworkflows combine them.

3.5. Workflow

```
workflows/
└── viralrecon.nf
    └── workflow VIRALRECON {...}
        └── FASTQ_TRIM_FASTP_FASTQC
(ch_cat_fastq)
        └── BAM_TRIM_PRIMERS_IVAR (ch_bam)


1 directory, 1 file
```

The logic that connects processes and subworkflows. A workflow can combine:

- Individual processes
- Subworkflows
- Modules
- Channels connecting them

A workflow defines how the components of an analysis are connected.

3.6. Pipeline

```
pipeline/
├── CHANGELOG.md
├── CITATIONS.md
├── conf/
├── docs/
├── main.nf
├── modules/
├── modules.json
├── nextflow.config
├── subworkflows/
├── tests/
└── workflows/
```

End-to-end workflows for a specific analysis. A pipeline provides:

- End-to-end analysis
- Standardised structure
- Configuration and parameters
- Documentation
- Tests
- Continuous integration
- Reproducible execution

3.7. nf-core tools

```
                                          ,--./,-.
          ___     __   __   __   ___     /,-._.--~\
    |\ | |__  __ /  ` /  \ |__) |__         } {
    | \| |       \__, \__/ |  \ |___     \`-._,-`-,
                                          `._,._,'

    nf-core/tools version 4.2.1 - https://nf-co.re


Usage: nf-core [OPTIONS] COMMAND [ARGS]...

nf-core/tools provides a set of helper tools for use with nf-core Nextflow pipelines.
It is designed for both end-users running pipelines and also developers creating new pipelines.

─ Commands ──────────────────────────────────────────────────────────────────────────────────
interface                          Launch the nf-core interface
modules          m,module          Commands to manage Nextflow DSL2 modules (tool wrappers).
pipelines        p,pipeline        Commands to manage nf-core pipelines.
subworkflows     s,swf,subworkflow Commands to manage Nextflow DSL2 subworkflows (tool
                                   wrappers).
test-datasets    t,td,tds,test-datasets Commands to manage nf-core test datasets.

─ Options ───────────────────────────────────────────────────────────────────────────────────
--version              Show the version and exit.
--verbose         -v   Print verbose output to the console.
--hide-progress        Don't show progress bars.
--log-file        -l   Save a verbose log to a file. [<filename>]
--help            -h   Show this message and exit.
```

The command-line toolkit for the nf-core ecosystem:

- Create new nf-core pipelines from a template
- Install and manage modules and subworkflows
- Validate and lint pipelines
- Generate and manage pipeline schemas
- Help with development and testing

HANDS-ON

Thanks for your attention!

BU-ISCIII

- Isabel Cuesta de la Plaza
- Sara Monzón Fernández
- Sarai Varona Fernández
- Víctor Manuel López Molina
- Juan Ledesma Moreno
- Alejandro Bernabéu Duréndez
- Pau Pascual Mas
- Magdalena Matito Silvero
- Olga Dolgova Konjushenko
- Enrique Sapena Ventura
- Ángela del Pozo Maté



CIBERINFEC BiPLAT
- Daniel Valle Millares
- Alba Talavera Rodríguez

Find us at
https://github.com/BU-ISCIII